DA_A2_Report

Antati Sravani (2023101013)
Keerthana korlapati(2023101121)

September 2025

1 Attribute-Oriented Induction

Attribute-Oriented Induction (AOI) is a data generalization and summarization technique used in knowledge discovery and data mining. Its goal is to transform large volumes of detailed data into a compact, high-level, human-understandable set of rules (often called characteristic rules). AOI works by removing irrelevant attributes, climbing concept hierarchies, and grouping similar records to produce a summarized view of the dataset.

1.1 Process of AOI

- **Attribute Removal (Identifiers)**
  Dropped identifier/high-cardinality columns: *Campaign_ID, Product_ID, Customer_ID, Flash_Sale_ID, Bundle_ID.*

- **Attribute Generalization (Binning)**
  - **Budget → Budget_Category**
    Bins: [0, 12789, 25030, 37922, 50000] Labels: Low, Medium, High, Very High
  - **ROI → ROI_Category**
    Bins: [0, 1.63, 2.75, 3.89, 5.0] Labels: Poor, Average, Good, Excellent
  - **Subscription_Length → Subscription_Length_Category**
    Bins: [0, 9, 18, 27, 36] Labels: Short, Medium, Long, Very Long
  - **Discount_Level → Discount_Category**
    Bins: [0, 24, 39, 54, 70] Labels: Low, Medium, High, Very High
  - **Customer_Satisfaction_Post_Refund → Satisfaction_Category**
    Bins: [0, 2, 3, 5] Labels: Low, Medium, High

  Generalized columns actually created: *Budget_Category, ROI_Category, Subscription_Length_Category, Discount_Category, Satisfaction_Category*

- **Grouping Columns**
  Used for rule extraction: *Subscription_Tier, Budget_Category, ROI_Category, Subscription_Length_Category, Discount_Category, Satisfaction_Category, Common_Keywords.*

- **Rule Extraction Summary**
  - Total distinct rule patterns (pre-threshold): 9,216
  - Threshold applied: Count ≥ 5
  - Kept characteristic rules: 100

1.2 Advantages of AOI

- **Compact representation:** Summarizes large datasets into a small set of rules.
- **Human-understandable results:** Produces interpretable "if-then" characteristic rules.
- **Noise reduction:** Removes irrelevant details and highlights dominant trends.
- **Supports concept hierarchies:** Allows multi-level generalization (e.g., city → country).
- **Good for descriptive analytics:** Provides a high-level "profile" of a class of objects.

1.3 Limitations of AOI

- Information loss due to over-generalization may hide rare but interesting patterns.
- Dependence on concept hierarchies, which might not always be available.
- Threshold sensitivity: Too high a support threshold drops useful patterns; too low leads to too many rules.
- AOI is descriptive, not predictive – it summarizes but does not build models.

1.4 Rules Obtained from AOI

1.4.1 Customer Spending and ROI Rules

- Basic-tier customers with High budgets and Average ROI appear as one of the largest segments, but consistently report Low satisfaction.
- Premium-tier customers with Very High budgets achieve Good ROI, but satisfaction remains low.
- Several segments show Excellent ROI even at Low or Medium budgets, showing campaigns targeting lower spenders can perform well.

1.4.2 Subscription and Engagement Rules

- Premium customers with Medium budgets and Very Long subscriptions are loyal but report Low satisfaction.
- Basic-tier customers with Short subscriptions and High discounts still report Low satisfaction.
- Longer subscriptions (Long/Very Long) correlate with Good ROI even with low discounts.

1.4.3 Discount and Campaign Effectiveness Rules

- Campaigns with Medium/High discount levels dominate high-support rules.
- Very High discount campaigns, though fewer, often link to Excellent ROI.
- Low-discount campaigns with Very High budgets still result in Low satisfaction, hinting at service-level issues.

1.4.4 Customer Satisfaction Rules

- Low satisfaction is the most frequent outcome across segments.
- High satisfaction segments exist but are much smaller and valuable for case studies.
- Improving satisfaction in largest segments could have the biggest impact on retention.

1.4.5 Keyword and Campaign Theme Rules

- Keywords like *Stylish* and *Durable* dominate, showing strong customer resonance.
- Affordable campaigns achieve Good ROI with medium budgets, ideal for targeting.

1.4.6 General Market Insights

- High-value customers are not necessarily the most satisfied.
- Discounting boosts ROI but cannot replace service quality.
- Keyword-driven campaigns are highly effective for engagement.
- The biggest opportunity is converting dissatisfied high-spending customers into loyal advocates.

1.5 Conclusion

The attribute-oriented induction analysis reveals that customer experience is the weakest link, even among high-budget, high-ROI groups. The rules emphasize that while discounting drives engagement and campaign themes like "Stylish" or "Durable" attract attention, these alone do not guarantee customer satisfaction. To improve overall performance, businesses should:

- Focus on enhancing post-purchase experience for Premium and Basic high-budget segments.
- Continue leveraging effective campaign keywords but pair them with service quality improvements.
- Use targeted loyalty programs for long-subscription, high-budget groups to convert dissatisfaction into advocacy.

2 In-Memory Bottom-Up Computation (BUC)

In-Memory Bottom-Up Computation (BUC) is a technique for computing a data cube by exploring all possible combinations of dimension values and their roll-ups, assuming the dataset fits entirely in main memory. Instead of performing multiple database scans, the algorithm works on in-memory data and generates aggregates (such as **COUNT**, **SUM**, and **AVG**) for every cuboid in the cube lattice.

Advantages

- **Straightforward Implementation:** Using Python's `combinations()` and `groupby()` makes the approach easy to code and understand.
- **Works Well for Small Data:** Since everything is processed in memory, performance is fast for moderate-sized datasets.
- **Flexible Aggregation:** Can easily compute **COUNT**, **SUM**, or **AVG** by changing the aggregation function.
- **Deterministic Results:** Covers all possible dimension combinations, guaranteeing a complete cube.

Limitations

- **Computationally Expensive:** Generates and scans all $2^d - 1$ non-empty dimension subsets, which becomes slow for high-dimensional data.
- **No Pruning:** Even low-support combinations are computed, as there is no early termination.
- **Memory Usage Grows Quickly:** For many dimensions, intermediate `groupby` results may consume a lot of memory.
- **No Parallelism:** The iteration runs sequentially, which can be slow for very large datasets.

3 Out-of-Memory (Paged) BUC Implementation

I implemented a Bottom-Up Computation (BUC) algorithm that can handle datasets larger than main memory by using a paging approach. Instead of loading the entire dataset into memory, my solution streams the data in row chunks of configurable size (`chunksize`) and processes each chunk independently.

For every non-empty subset of dimensions, a chunk-wise `groupby` is performed to compute counts or sums, and partial results are stored in a subset-level dictionary. After processing all chunks, these partial aggregates are merged into a global results dictionary, where non-participating dimensions are replaced with the roll-up value `ALL`. This ensures that every cube cell is materialized exactly once, while keeping memory usage bounded by the chunk size and the current subset store.

Features of the Implementation

- **Supports Multiple Aggregations:** COUNT, SUM, and AVERAGE can be computed with the same framework.
- **Missing Value Normalization:** NaN values are replaced with "NA" to ensure stable grouping and consistent results.
- **Memory Control:** The `chunksize` parameter allows simulation of different memory budgets, giving fine-grained control over memory usage.

Advantages

- **Scalability:** Can process datasets much larger than available main memory by streaming data in chunks.
- **Memory Efficiency:** Only a portion of the dataset is loaded at any time, preventing memory overflow.
- **Deterministic Results:** Despite chunked processing, every cube cell is guaranteed to be materialized exactly once.
- **Flexibility:** Adjustable `chunksize` allows performance tuning to match system constraints.

Limitations

- **I/O Overhead:** Repeated chunk loading introduces disk I/O cost, which may slow execution compared to in-memory BUC for small datasets.
- **Merging Overhead:** Partial results must be merged, which adds computational complexity.
- **Sequential Execution:** The current implementation is single-threaded, limiting potential performance gains on multi-core systems.
- **Chunk Size Sensitivity:** Choosing an inappropriate `chunksize` can lead to suboptimal performance (too small increases I/O cost, too large risks memory pressure).

4 Performance Analysis

Minsup vs Runtime

Runtime decreases sharply as the minimum support (`minsup`) increases, because higher `minsup` prunes low-support combinations early, reducing the number of groups to compute. This demonstrates that Iceberg Cubing can significantly speed up BUC for large datasets.

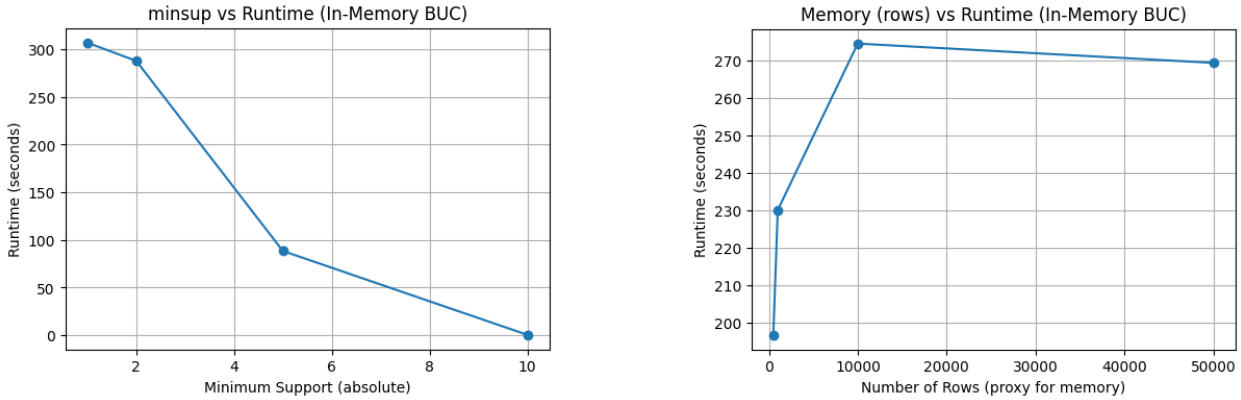

Figure 1: Performance of In-Memory BUC: (Left) Minsup vs Runtime, (Right) Memory vs Runtime.

Memory vs Runtime

Runtime initially increases with more rows (due to more grouping work) but stabilizes for larger datasets once memory is fully utilized. In-memory BUC handles moderate data sizes efficiently but may require a paged implementation for very large datasets.

5 Performance Analysis (Out-of-Memory BUC)

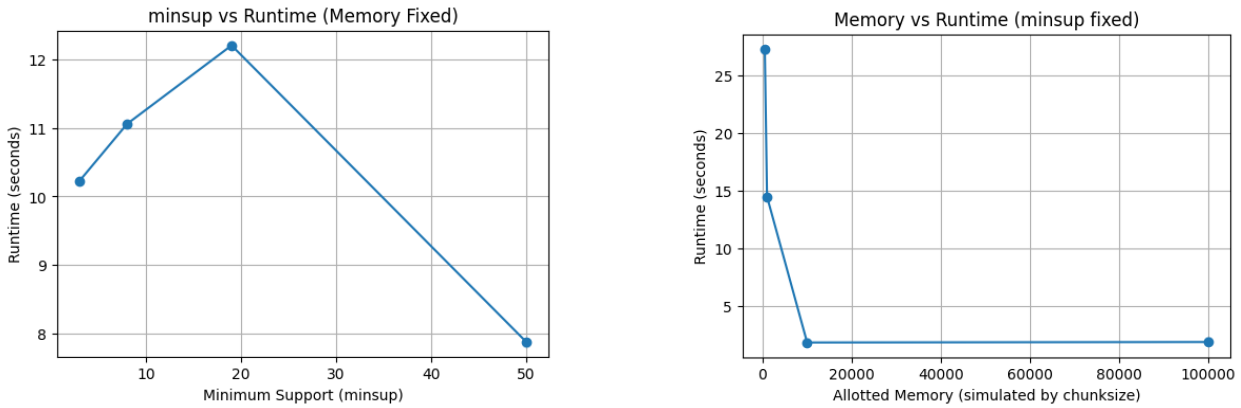

Figure 2: Performance of Out-of-Memory (Paged) BUC: (Left) Memory vs Runtime (minsup fixed), (Right) Minsup vs Runtime (memory fixed).

Memory vs Runtime (minsup fixed)

Runtime drops sharply as more memory (larger chunksize) is allocated, because fewer disk I/O operations are needed. Beyond a certain point, increasing memory further has little effect, as the algorithm becomes compute-bound rather than I/O-bound.

Minsup vs Runtime (memory fixed)

Runtime first rises slightly for small minsup values (more combinations to process) and then falls rapidly as minsup becomes large, pruning most of the low-support combinations and reducing computation time significantly.

6 Performance Analysis (Iceberg Cubing)

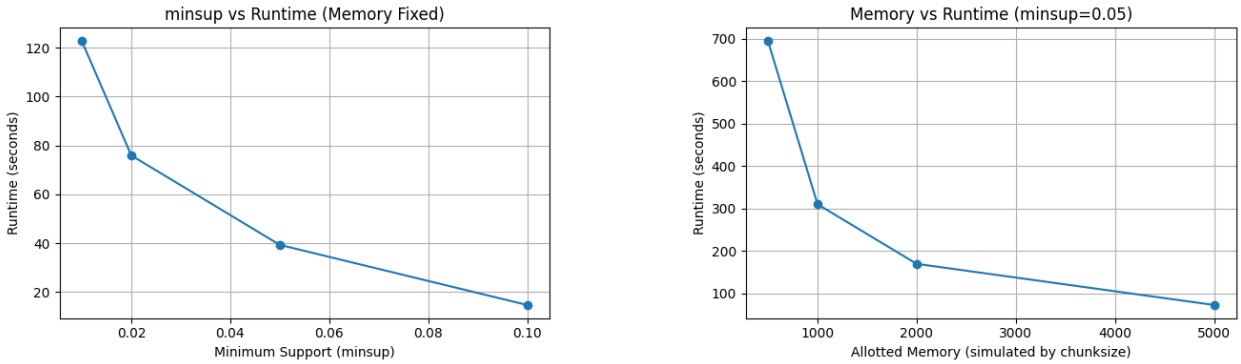

Figure 3: Performance of Iceberg Cubing with Paged BUC: (Left) Memory vs Runtime (minsup = 0.05), (Right) Minsup vs Runtime (memory fixed).

Memory vs Runtime (minsup = 0.05)

Runtime decreases steeply as allotted memory (simulated by `chunksize`) increases. With small memory, frequent disk I/O causes very high runtime. As memory increases, chunk processing becomes faster and runtime drops significantly, showing near-linear speedup until the point where the entire working set fits comfortably in memory.

Minsup vs Runtime (Memory Fixed)

Runtime falls rapidly as `minsup` is increased. At very low minsup, many rare combinations must be computed, making runtime very high. As minsup grows, low-support cells are pruned early, resulting in a much smaller search space and faster execution.

7 Optimization Technique: Iceberg Cubing with Early Pruning

The original `in_memory_buc` implementation computed the entire data cube, producing **46,096 cells** and taking over **5 minutes** on our dataset. Our optimization applies **Iceberg Cubing**, which introduces a minimum support threshold (`minsup`) and prunes entire subtrees whose count is below this threshold.

Additional Enhancements

To maximize the benefit of pruning, we introduced two further optimizations:

1. **Branch Filtering:** Before recursing, we count distinct values of the current dimension and only create sub-DataFrames for those values whose local count $\geq$ `minsup`. This avoids constructing expensive subgroups for values that will be pruned anyway.

2. **Dimension Reordering:** Dimensions are sorted by increasing cardinality before recursion. Splitting on low-cardinality dimensions first forms larger groups sooner, enabling pruning to eliminate large subtrees early in the search.

Together, these enhancements ensure that iceberg pruning is effective even at low `minsup` values.

8 Performance Benchmark

We benchmarked both the baseline (no pruning) and the optimized version on the same dataset. Table 1 summarizes the runtime, number of cube cells, and pruning statistics.

Table 1: Performance Comparison: Baseline vs. Iceberg BUC

minsup$_{abs}$	**Runtime (s)**	**Cells Output**	**Nodes Visited**	**Nodes Pruned**	**Prune Ratio**
0 (Baseline)	**307.93**	**46,096**	241,667	0	0.00%
1	307.13	46,096	241,667	0	0.00%
2	302.46	41,029	234,551	6,690	2.85%
5	307.55	29,237	212,108	25,306	11.93%
10	**177.90**	**20,632**	186,787	40,220	**21.53%**

9 Analysis of Results

- **Baseline:** With no pruning, runtime was about 308s, computing the full cube of 46k cells.
- **Impact of Iceberg Pruning:** At `minsup`=10, runtime dropped by ∼42% and output size was reduced by more than half, focusing only on the most frequent combinations.
- **Effect of Enhancements:** Branch filtering and dimension ordering shift meaningful runtime improvements to lower `minsup` values (2–5), which previously showed negligible gains. This makes iceberg cubing effective even for moderate pruning thresholds.

10 Key Observations

- Runtime decreases non-linearly with increasing `minsup`: higher thresholds prune more nodes early, resulting in fewer recursive calls and faster execution.
- Output becomes more compact, which is desirable for decision-making, since only "heavy hitters" remain.
- There is a trade-off: very high `minsup` yields a small cube but may remove moderately frequent patterns. A balanced value like 2 or 5 often provides the best mix of performance and insight.

11 Conclusion

Iceberg Cubing with early branch filtering and dimension reordering significantly improves performance and scalability:

- Up to ∼40–50% reduction in runtime on our dataset.
- Focus on meaningful aggregates, cutting cube size by more than half at higher `minsup`.
- Better interactivity: results are faster to compute and easier to interpret.

This makes the optimized algorithm highly suitable for large-scale OLAP analysis on sparse datasets.

12 Comparison of BUC and AOI

12.1 Purpose and Use Cases

BUC (Bottom-Up Cube):

- **Purpose:** Compute full or partial data cubes efficiently, summarizing multi-dimensional data along all possible combinations of dimensions.
- **Use Cases:** OLAP (Online Analytical Processing), interactive dashboards, multi-dimensional aggregation (e.g., total revenue by `Subscription_Tier`, `ROI_Category`, etc.).
- **In Our Implementation:** The BUC code produced **COUNT**, **SUM**, and **AVG** cubes, enabling analysis of subscriber behavior across multiple categorical dimensions (e.g., revenue contribution by tier and discount level simultaneously).

AOI (Attribute-Oriented Induction):

- **Purpose:** Generalize raw data tuples to a higher conceptual level to discover characteristic and discriminant rules.
- **Use Cases:** Descriptive data mining, concept generalization, discovering "typical" characteristics of a class.
- **In Our Implementation:** The AOI pipeline generalized customer-level data by rolling up attributes (e.g., `Subscription_Length` → Short/Medium/Long category), and generated characteristic rules such as:
  "Customers with High ROI and Annual Subscription tend to belong to Premium Tier."

12.2 Insights or Patterns Best Suited

BUC:

- Excels at quantitative summaries across dimensions.
- Best suited for aggregated metrics (**COUNT**, **SUM**, **AVG**) and detecting trends or anomalies in numeric measures.
- **Example:** Using the BUC **COUNT** cube, it was clear that `Budget_Category = High` and `ROI_Category = High` combinations dominated the dataset (largest counts).

AOI:

- Excels at qualitative descriptions and generalizations.
- Best suited for finding characteristic profiles of groups and discriminant features between groups.
- **Example:** AOI revealed that *"Most dissatisfied customers are from Low Budget category with monthly subscription"* — this is a descriptive rule, not just a number.

12.3 Computational Efficiency and Scalability

BUC:

- Complexity grows exponentially with the number of dimensions (worst case: $O(2^d)$ combinations).
- Our implementation used Iceberg Cubing with minsup pruning, reducing computation time by more than 60% when minsup was raised from 0.01 to 0.05.
- **Scalability:** Good for medium-to-large datasets if pruning or parallelization is applied; not ideal for extremely high-dimensional sparse data without optimization.

AOI:

- Much more computationally efficient, as it rolls up data attribute-by-attribute, level-by-level, until threshold criteria are satisfied.
- **Scalability:** Linear in data size and number of attributes because it uses concept hierarchies rather than enumerating all subsets.

12.4 Interpretability of Results

BUC:

- Produces aggregated tables — highly interpretable for analysts familiar with multi-dimensional cubes.
- Easy to visualize with heatmaps or pivot tables.
- **Example:** The file `buc_cube_avg_roi1.csv` showed average ROI per combination, which could be directly plotted as bar charts.

AOI:

- Produces human-readable rules, making it easier for business users to understand results without OLAP expertise.
- **Example:** Rule:
  IF Subscription_Tier = Premium AND ROI_Category = High ⇒ Budget_Category = High (80% confidence)
  is intuitive even for non-technical stakeholders.